

Musterlösung — Übungsklausur 6

Deep Learning 1 · TU Berlin

1 Multiple Choice

1. Vanishing Gradients RNN?

✓ (b) Entstehen wenn größter Eigenwert der versteckten Gewichtsmatrix < 1

- (a) Eigenwert $> 1 \rightarrow$ Exploding
- (c) Kurze Sequenzen
- (d) Vorwärtspropagation

2. Self-Supervised Learning?

✓ (b) Künstliche Aufgabe mit automatisch erzeugten Labels liefert nützliche Features

- (a) Modell labelt selbst
- (c) Training ohne Labels
- (d) Modelle trainieren sich gegenseitig

3. Conservation Prinzip?

✓ (b) $\sum \phi_i = f(x)$ — Attributionen summieren zum Modelloutput

- (a) Alle positiv
- (c) Über Schichten gemittelt
- (d) Gleiche Werte

4. Sparse Coding vs. PCA?

✓ (b) Sparse Coding $\rightarrow$ spärliche Repräsentation; PCA $\rightarrow$ orthogonale Achsen maximaler Varianz

- (a) Umgekehrt
- (c) Identisch
- (d) PCA rechneintensiver

5. UNets?

✓ (b) Nutzen Skip Connections: Low-Level-Information direkt an Decoder

- (a) Keine Skip Connections
- (c) Zeitreihendaten
- (d) Kein Pooling

6. Multi-Task Loss?

✓ (b) Gewichtung der Task-Losses ist wichtiger Hyperparameter

- (a) Keine verschiedenen Losses
- (c) Immer schlechter
- (d) Nicht gemeinsam trainierbar

7. GRU?

✓ (a) Variante des LSTM mit zusammengefassten Input- und Forget-Gates

- (b) CNN für Zeitreihen
- (c) Gradientenmethode
- (d) Regularisierung

8. Convolutional Sparse Coding?

✓ (a) Sparse Coding mit Convolutional Filtern als Wörterbuch

- (b) Auf 1D Signalen
- (c) PCA-Variante
- (d) Keine korrekte Antwort

2 Theorie — LRP

(a) Kernidee LRP

LRP propagiert die Ausgabe-Relevanz rückwärts durch das Netz, Schicht für Schicht, unter Verwendung der Netzwerkstruktur. Relevanz wird auf Eingabefeatures aufgeteilt, sodass die Summe erhalten bleibt (Conservation).

Unterschied zu Gradienten: Gradienten messen lokale Sensitivität (wie empfindlich ist Output auf kleine Änderungen). LRP misst Attribution (welcher Feature hat wie viel zum Output beigetragen) — global, nicht lokal.

(b) Vor- und Nachteile LRP

Vorteile: Gute Erklärungsqualität auf tiefen Netzen. Schnell (~1 Forward-/Backward-Pass). Flexibel in der Propagationsregel.

Nachteile: Hyperparameter der Propagationsregeln müssen gewählt werden. Muss für jede neue Architektur angepasst werden. Nimmt an, dass das Modell die Funktion ausreichend disentangled hat.

(c) Attribution vs. Activation Maximization

Attribution/Heatmap: Erklärt EINE spezifische Vorhersage $f(x)$ für einen konkreten Datenpunkt x . Zeigt, welche Eingabefeatures zu dieser spezifischen Vorhersage beigetragen haben.

Activation Maximization: Erklärt, was das Modell GLOBAL gelernt hat — sucht synthetischen Input der ein Neuron maximal aktiviert. Ungeeignet für Erklärung einzelner Vorhersagen.

3 Theorie — Datenzentrierung & Xavier

(a) Vorteile/Nachteile zentrierter Daten

Vorteile: Konditionszahl der Hessematrix wird 1 (Eigenwerte gleichmäßig) → schnellere Konvergenz von Gradientenabstieg, geringerer Vorhersagefehler.

Nachteile: Trainingsmittelwert muss gespeichert werden. Testdaten müssen mit Trainingsmittelwert vorverarbeitet werden.

(b) Xavier-Herleitung

Annahmen: $E[W_{ij}] = 0$, Inputs $z^{(l)}$ unkorreliert, $\text{Var}(W_{ij}) = \eta$.

$$\text{Var}(z^{(l+1)}_i) = \sum_j E[(W_{ij})^2] \cdot \text{Var}(z^{(l)}_j) = n \cdot \eta \cdot \text{Var}(z^{(l)}_j)$$

Für $\text{Var}(z^{(l+1)}) = \text{Var}(z^{(l)})$: setze $\eta = 1/n$.

$$w^{(l)}_{ij} \sim N(0, 1/n_l) \text{ mit } n_l = \text{Inputdimension der Schicht } l$$

4 Theorie — Conditional RBMs

(a) Energiefunktion cRBM

$$E(x, h, y) = -x^\top W h - y^\top U h$$

x : Input, h : latente binäre Variablen, y : Output. W , U : Gewichtsmatrizen.

(b) Marginalisierung & freie Energie

$$p(x, y) = Z^{-1} \cdot \exp(-F(x, y))$$

$$F(x, y) = -\sum_k \log(1 + \exp(w_{:,k}^\top x + u_{:,k}^\top y))$$

Die freie Energie entspricht einem zweischichtigen neuronalen Netz mit Sigmoid-Aktivierung.
Marginalisierung über h eliminiert die latenten Variablen analytisch.

(c) Training cRBM

$$\max_W (1/N) \sum_n \log p(x^n, y^n)$$

Gradientenberechnung: $\partial/\partial W_{ik} = E_{\{p\}}[x_i \cdot \sigma(W_{ik}x)] - E_{\{p(x,y)\}}[x_i \cdot \sigma(W_{ik}x)]$

Linke Seite: Erwartung unter Datenmenge (einfach berechenbar). Rechte Seite: Erwartung unter Modell (approximiert via MCMC/Contrastive Divergence).

5 Programmierung — U-Net Segmentierung

(a) U-Net Architektur & Skip Connections

U-Net: Encoder (Downsampling) + Decoder (Upsampling) mit Skip Connections, die Feature Maps aus dem Encoder direkt in den entsprechenden Decoder-Level einspeisen.

Bedeutung für Segmentation: Low-Level-Features (Kanten, Texturen) → fließen direkt via Skip zum Decoder → präzise Segmentierungsgrenzen. High-Level-Features → im Bottleneck → Verständnis des Segmentinhalts. Beide nötig für gute Segmentierung.

(b) Klassifikation vs. Segmentierung

Klassifikation: Ein Label pro Bild. Segmentierung: Ein Label pro Pixel. Loss: Pixel-weiser Cross-Entropy oder Dice Loss.

(c) U-Net Pseudo-Code

```
def forward(x):
    e1 = encoder_block1(x) # z.B. 256→128
    e2 = encoder_block2(e1) # 128→64
    b = bottleneck(e2) # 64→64
    d1 = decoder_block1(b, e2) # concat skip von e2
    d2 = decoder_block2(d1, e1) # concat skip von e1
    return final_conv(d2) # Pixel-Klassen
```